MATH 345 Skeleton Notes

Unit 3: Beyond linear maps: local linearization and first-order optimization

Section 3.5: Chain rule as matrix multiplication

© 2024–2026 Jacob Denson, Kaitlyn Phillipson, Sebastien Roch. Except where otherwise noted, this work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Unit 1 used matrix multiplication to represent composition of linear maps: applying B first and then A gives $A(B\mathbf{x}) = (AB)\mathbf{x}$. For differentiable nonlinear maps, the same idea reappears locally. The local linear maps compose, so their Jacobian matrices multiply.

The chain rule

The chain rule describes derivatives of composite functions.

Compositions

Definition: Composite Function

If $G : \mathbb{R}^n \to \mathbb{R}^p$ and $F : \mathbb{R}^p \to \mathbb{R}^m$ are multivariable functions, we can form a new composite function as follows. Take an input in $\mathbb{R}^n$, and then apply G to the input to obtain a vector in $\mathbb{R}^p$. Apply F to this vector to obtain a vector $\mathbb{R}^m$. This is the output of the composite function. In other words, our input is fed through the following diagram

$$\mathbb{R}^n \xrightarrow{G} \mathbb{R}^p \xrightarrow{F} \mathbb{R}^m.$$

We write the composite function as $F \circ G$ (the symbol $\circ$ should be read as "composed with"). In symbols, the composite function is defined as

$$(F \circ G)(\mathbf{x}) = F(G(\mathbf{x})).$$

Note

Note that the composite $F \circ G$ is *only* defined when the output of G is a possible input to F, i.e., the output of G is $\mathbb{R}^p$, which is the space of inputs to F.

Activity: Composing two matrix maps (U1-LO5, U1-LO6)

Let $T_1 : \mathbb{R}^2 \to \mathbb{R}^3$ and $T_2 : \mathbb{R}^3 \to \mathbb{R}^2$ be linear maps defined by their component functions as follows:

$$T_1(x, y) = \begin{bmatrix} 2x - y \\ 3y \\ x + y \end{bmatrix} \qquad T_2(u, v, w) = \begin{bmatrix} u + v - w \\ 2u - v \end{bmatrix}.$$

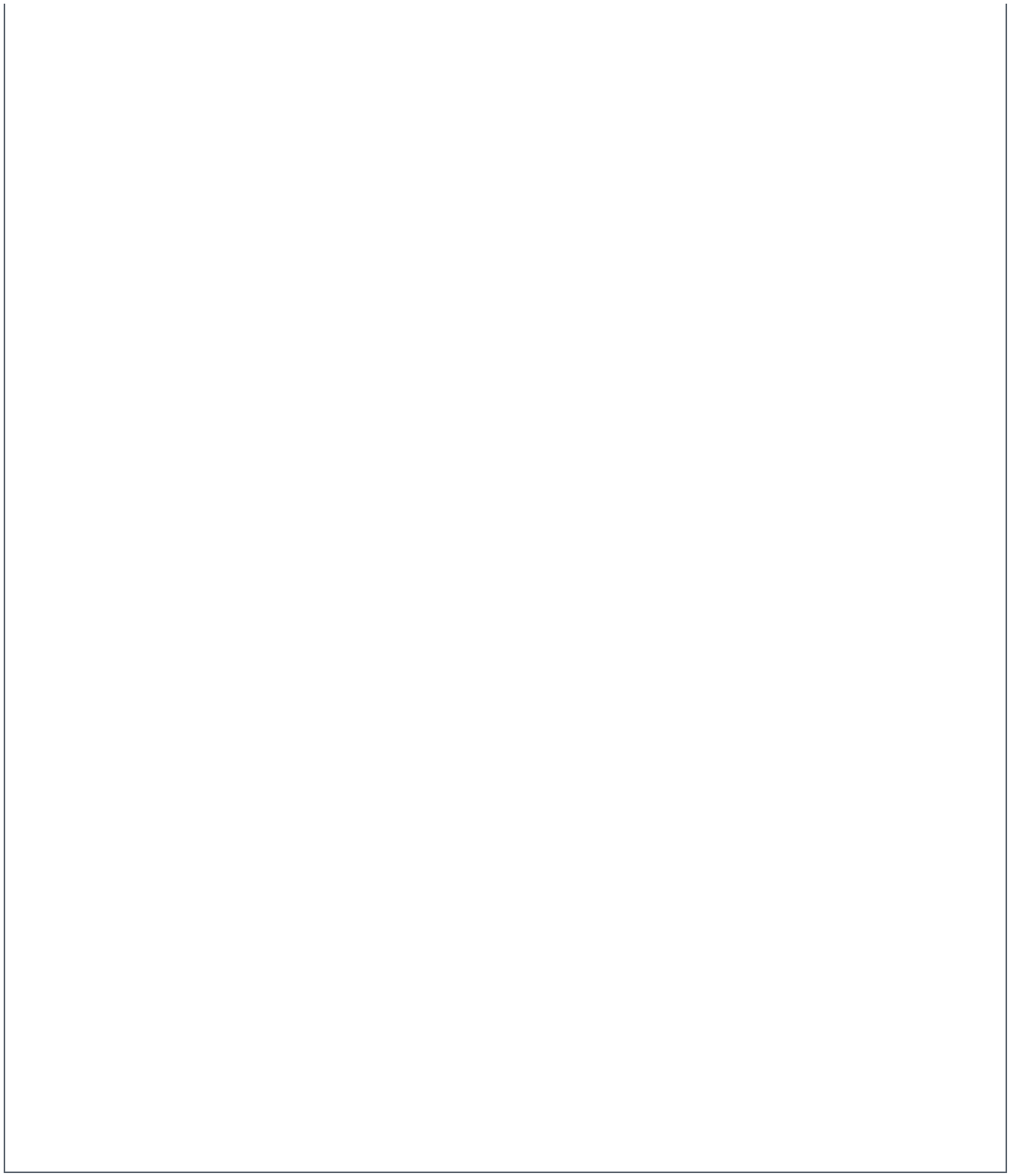
Find the matrices A_1 and A_2 associated with these maps, so that $T_1 = T_{A_1}$ and $T_2 = T_{A_2}$ (recall the notation from Matrix map).

Activity: Composing two matrix maps (U1-LO5, U1-LO6)

Let $T_1 : \mathbb{R}^2 \to \mathbb{R}^3$ and $T_2 : \mathbb{R}^3 \to \mathbb{R}^2$ be linear maps defined by their component functions as follows:

$$T_1(x, y) = \begin{bmatrix} 2x - y \\ 3y \\ x + y \end{bmatrix} \qquad T_2(u, v, w) = \begin{bmatrix} u + v - w \\ 2u - v \end{bmatrix}.$$

Calculate the composite map $T_2 \circ T_1$ and express it as a matrix map from $\mathbb{R}^2$ to $\mathbb{R}^2$.



Here is a nonlinear example of the composition of two functions.

Activity: Composing nonlinear maps in both orders (U3-LO1)

Consider $F : \mathbb{R}^2 \to \mathbb{R}^2$ and $G : \mathbb{R}^2 \to \mathbb{R}^2$ defined by

$$F(u, v) = (uv, u + v) \quad \text{and} \quad G(x, y) = (e^{xy}, x - y).$$

Find expressions for $(F \circ G)(x, y)$ and $(G \circ F)(u, v)$.

Warning

Observe that $F \circ G$ and $G \circ F$ are very different functions. We must therefore be careful that we compose functions in the correct order.

The order matters. The rightmost Jacobian matrix acts first on the input change.

Activity: Composition with a parametrized curve (U3-LO1)

Consider $\mathbf{r} : \mathbb{R} \to \mathbb{R}^3$ and $\mathbf{p} : \mathbb{R}^3 \to \mathbb{R}^2$ defined by

$$\mathbf{r}(t) = (t, \cos(t), \sin(t)) \quad \text{and} \quad \mathbf{p}(x, y, z) = (y, z)$$

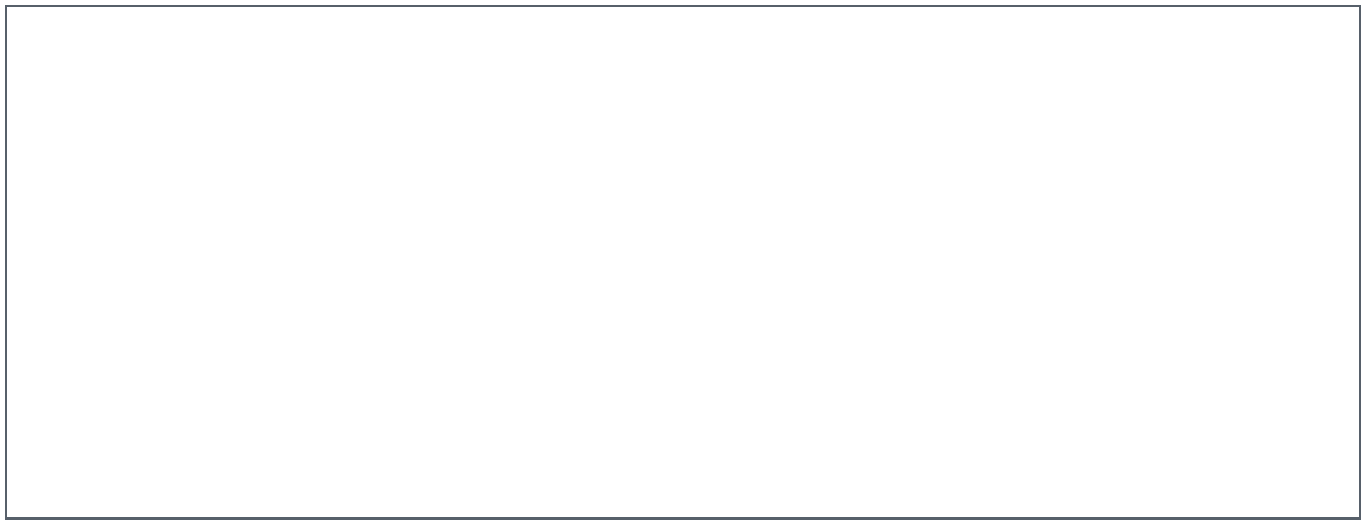
Find an expression for $(\mathbf{p} \circ \mathbf{r})(t)$.

Activity: Composition with a parametrized curve (U3-LO1)

Consider $\mathbf{r} : \mathbb{R} \to \mathbb{R}^3$ and $\mathbf{p} : \mathbb{R}^3 \to \mathbb{R}^2$ defined by

$$\mathbf{r}(t) = (t, \cos(t), \sin(t)) \quad \text{and} \quad \mathbf{p}(x, y, z) = (y, z)$$

Is the composition $\mathbf{r} \circ \mathbf{p}$ defined? Why or why not?



Chain rule for one-parameter inputs

Recall the chain rule from single-variable calculus, that for two functions $f : \mathbb{R} \to \mathbb{R}$ and $g : \mathbb{R} \to \mathbb{R}$, we have

$$(f \circ g)'(t) = (f' \circ g)(t)g'(t).$$

The next theorem gives the chain rule in component and gradient forms. Recall that for a function $f(x, y)$, $f_x = \frac{\partial f}{\partial x}$, and that $f_y = \frac{\partial f}{\partial y}$.

Theorem: Chain Rule for One Independent Variable

Consider a scalar-valued function $f : \mathbb{R}^2 \to \mathbb{R}$, and a vector-valued function $\mathbf{r} : \mathbb{R} \to \mathbb{R}^2$ of a single variable. Then the composition $f \circ \mathbf{r} : \mathbb{R} \to \mathbb{R}$ is a scalar-valued function of a single variable. If $\mathbf{r}$ is differentiable at t_0, and f is differentiable at $\mathbf{r}(t_0)$, then $f \circ \mathbf{r}$ is differentiable at t_0 and

$$(f \circ \mathbf{r})'(t_0) = f_x(\mathbf{a})r_1'(t_0) + f_y(\mathbf{a})r_2'(t_0) = \nabla f(\mathbf{a}) \cdot \mathbf{r}'(t_0),$$

where $\mathbf{a} = \mathbf{r}(t_0)$, and r_1 and r_2 are the components of $\mathbf{r}$.

Why is this true?

Let $\mathbf{a} = \mathbf{r}(t_0)$. For a small scalar change h in the parameter, the local linear approximation of the curve from Linear approximation in the one-parameter case gives

$$\mathbf{r}(t_0 + h) \approx \mathbf{a} + \mathbf{r}'(t_0)h.$$

Thus the change in the input to f is approximately $\mathbf{r}'(t_0)h$.
As explained in the proof of Continuity of First Partials Implies Differentiability, the local linear approximation of f near $\mathbf{a}$ is

$$f(\mathbf{a} + \mathbf{k}) \approx f(\mathbf{a}) + \nabla f(\mathbf{a}) \cdot \mathbf{k}.$$

Substitute the actual input change $\mathbf{k} = \mathbf{r}(t_0 + h) - \mathbf{a}$, then use the approximation from the curve:

$$\begin{aligned}(f \circ \mathbf{r})(t_0 + h) &\approx f(\mathbf{a}) + \nabla f(\mathbf{a}) \cdot \big(\mathbf{r}(t_0 + h) - \mathbf{a}\big) \\ &\approx f(\mathbf{a}) + \big(\nabla f(\mathbf{a}) \cdot \mathbf{r}'(t_0)\big)h.\end{aligned}$$

We have obtained the local affine approximation

$$L(t) = f(\mathbf{a}) + \big(\nabla f(\mathbf{a}) \cdot \mathbf{r}'(t_0)\big)(t - t_0).$$

Its linear part multiplies the parameter change by $\nabla f(\mathbf{a}) \cdot \mathbf{r}'(t_0)$. Reading off this coefficient gives

$$(f \circ \mathbf{r})'(t_0) = f_x(\mathbf{a})r_1'(t_0) + f_y(\mathbf{a})r_2'(t_0) = \nabla f(\mathbf{a}) \cdot \mathbf{r}'(t_0).$$

Differentiability ensures that the neglected errors are smaller than first order in h, so they do not change this coefficient.

Activity: Differentiating a function along a curve (U3-LO6)

If $f(x, y) = x^2 + y^2$, and $\mathbf{r}(t) = (\sin t, \cos(2t))$, find $(f \circ \mathbf{r})'(t)$.

Note

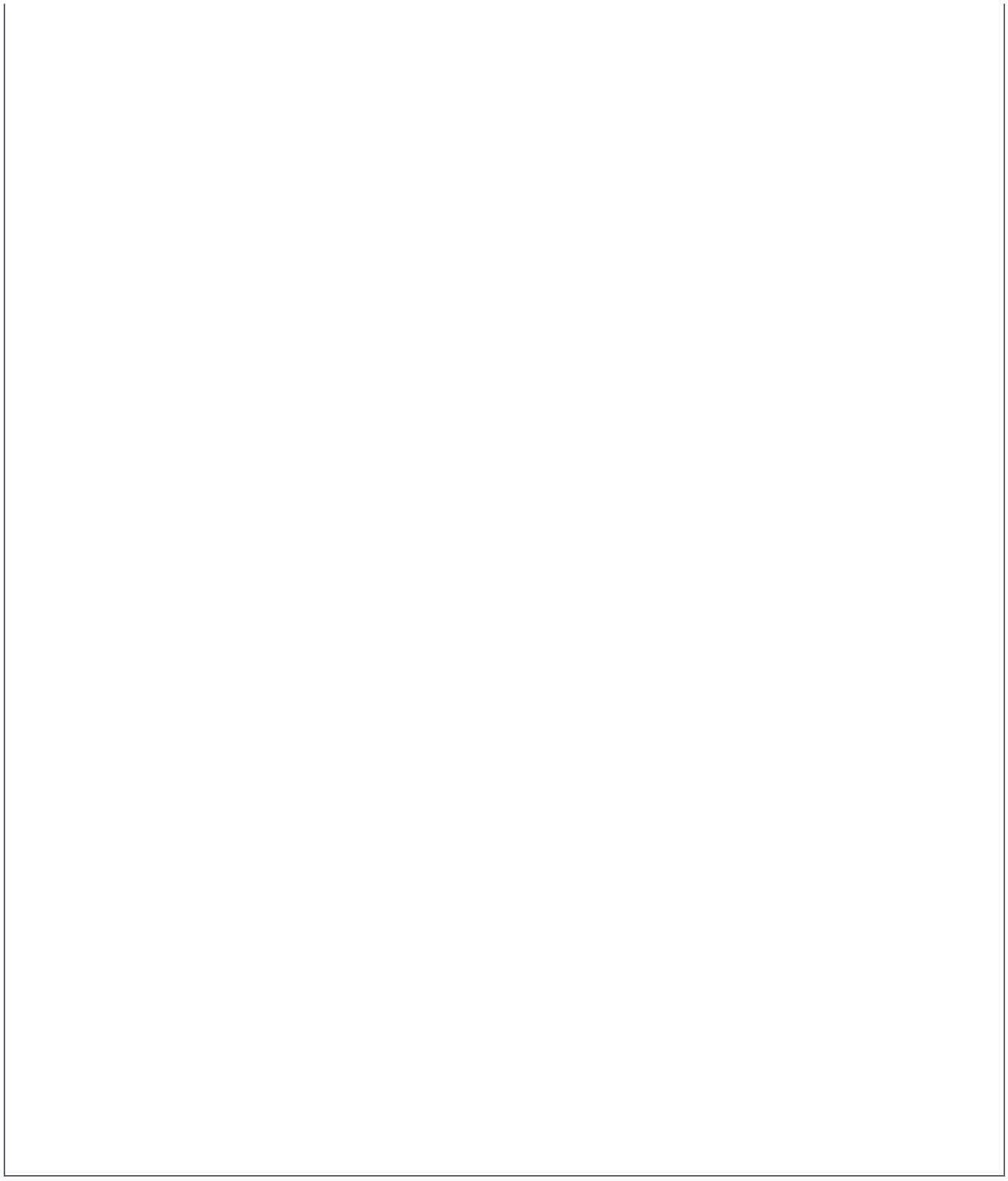
A useful mnemonic comes from writing $z = f(x, y)$, with $x = x(t)$ and $y = y(t)$. In this notation, the chain rule reads

$$\frac{dz}{dt} = \frac{\partial z}{\partial x}\frac{dx}{dt} + \frac{\partial z}{\partial y}\frac{dy}{dt},$$

so that the symbols dx and dy in the numerators on the right hand side, and the symbols ∂x and ∂y that occur in the denominators appear to 'cancel' like when multiplying fractions.

Activity: Differentiating through two intermediate variables (U3-LO6)

Suppose $z = x^2y - y^2x$, where $x = \sin t$ and $y = e^t$. Compute dz/dt.



Chain rule for two independent variables

Theorem: Chain Rule for Two Independent Variables

Suppose $f : \mathbb{R}^2 \to \mathbb{R}$ is a scalar-valued function, and $G : \mathbb{R}^2 \to \mathbb{R}^2$ is a vector-valued function. Then $f \circ G : \mathbb{R}^2 \to \mathbb{R}$ is a scalar-valued function, and the two equations

$$(f \circ G)_x = (f_x \circ G)(G_1)_x + (f_y \circ G)(G_2)_x$$

and

$$(f \circ G)_y = (f_x \circ G)(G_1)_y + (f_y \circ G)(G_2)_y$$

hold.

Why is this true?

Fix a base point $\mathbf{a} = (x_0, y_0)$ and write $\mathbf{b} = G(\mathbf{a}) = (b_1, b_2)$. For a small input change $\mathbf{h} = (h_1, h_2)$, use Componentwise linear approximation to approximate each component of G:

$$G_i(\mathbf{a} + \mathbf{h}) \approx b_i + (G_i)_x(\mathbf{a})h_1 + (G_i)_y(\mathbf{a})h_2, \qquad i = 1, 2.$$

The actual change in the input to f is $\mathbf{k} = G(\mathbf{a} + \mathbf{h}) - \mathbf{b}$. Near $\mathbf{b}$, the local linear approximation of f gives

$$f(\mathbf{b} + \mathbf{k}) \approx f(\mathbf{b}) + f_x(\mathbf{b})k_1 + f_y(\mathbf{b})k_2.$$

Substitute the approximations for k_1 and k_2 from the two components of G, and collect the terms in h_1 and h_2:

$$\begin{aligned}(f \circ G)(\mathbf{a} + \mathbf{h}) \approx f(\mathbf{b}) &+ \big[f_x(\mathbf{b})(G_1)_x(\mathbf{a}) + f_y(\mathbf{b})(G_2)_x(\mathbf{a})\big]h_1 \\ &+ \big[f_x(\mathbf{b})(G_1)_y(\mathbf{a}) + f_y(\mathbf{b})(G_2)_y(\mathbf{a})\big]h_2.\end{aligned}$$

This is an affine approximation: the constant term is $f(\mathbf{b}) = (f \circ G)(\mathbf{a})$, and the remaining terms are linear in the input change. The coefficient

of h_1 gives the partial derivative with respect to the first input, and the coefficient of h_2 gives the partial derivative with respect to the second input. Reading off these coefficients gives

$$(f \circ G)_x(\mathbf{a}) = f_x(\mathbf{b})(G_1)_x(\mathbf{a}) + f_y(\mathbf{b})(G_2)_x(\mathbf{a}),$$
$$(f \circ G)_y(\mathbf{a}) = f_x(\mathbf{b})(G_1)_y(\mathbf{a}) + f_y(\mathbf{b})(G_2)_y(\mathbf{a}).$$

Since $\mathbf{b} = G(\mathbf{a})$ and the base point is arbitrary, these are the two formulas in the theorem. Differentiability ensures that the neglected errors are smaller than first order in $\mathbf{h}$, so they do not change the linear part.

Remark

Recall from Partial derivatives, gradients, Hessians, and Jacobian matrices that the Jacobian matrix collects the partial derivatives of a vector-valued map. For G, this gives

$$J_G = \begin{bmatrix} \dfrac{\partial G_1}{\partial x} & \dfrac{\partial G_1}{\partial y} \\ \dfrac{\partial G_2}{\partial x} & \dfrac{\partial G_2}{\partial y} \end{bmatrix}.$$

Then we can write the pair of equations in Chain Rule for Two Independent Variables as the single vector equation

$$\nabla(f \circ G) = J_G^T(\nabla f \circ G),$$

where we interpret ∇f as a column vector. Transposing and using $J_f = (\nabla f)^T$ gives the equivalent Jacobian form

$$J_{f \circ G} = (J_f \circ G) J_G.$$

Activity: The chain rule as a Jacobian product (U3-LO6)

Use the Jacobian matrix version of the chain rule to calculate the partial derivatives of the function $f \circ G$, where $f(x, y) = 3x^2 - 2xy + y^2$, and $G(u, v) = (3u + 2v, 4u - v)$.

Activity: Checking the chain rule in SymPy (U3-LO6)

Revisit $f(x, y) = 3x^2 - 2xy + y^2$ and $G(u, v) = (3u + 2v, 4u - v)$ from the preceding calculation. The code computes $J_{f \circ G}$ in two ways: multiplying the Jacobian matrices and differentiating the composite expression directly. As in Checking the Jacobian of a gradient in SymPy (U3-LO3), we put a scalar expression in a one-entry matrix to use `.jacobian()`. The command `sp.simplify` simplifies symbolic expressions, including the entries of a matrix.

```
import sympy as sp

x, y, u, v = sp.symbols("x y u v")
f = 3*x**2 - 2*x*y + y**2
G = sp.Matrix([3*u + 2*v, 4*u - v])

Jf = sp.Matrix([f]).jacobian([x, y])
```

Activity: Checking the chain rule in SymPy (U3-LO6) (continued)

```
JG = G.jacobian([u, v])
Jf_at_G = Jf.subs({x: G[0], y: G[1]})
chain = sp.simplify(Jf_at_G * JG)

f_of_G = f.subs({x: G[0], y: G[1]})
direct = sp.Matrix([f_of_G]).jacobian([u, v])

print(Jf_at_G.shape, JG.shape, chain.shape)
print(chain)
print(sp.simplify(direct - chain))
print(chain.subs({u: 1, v: 0}))
```

Output:

```
(1, 2) (2, 2) (1, 2)
Matrix([[38*u + 18*v, 18*u + 34*v]])
Matrix([[0, 0]])
Matrix([[38, 18]])
```

1. Explain the three shapes in the first output line. Using the composition rule from Matrix multiplication as composition, explain why `JG` is the right-hand factor. Would the reversed product `JG * Jf_at_G` be defined?
2. Why do we substitute the components of `G` into `Jf`? At $(u, v) = (1, 0)$, where must the outer Jacobian be evaluated? Verify the last output line by multiplying the two evaluated matrices.
3. What does the zero matrix in the third output line establish for these functions? How does the row in the second output line relate to the column gradient $\nabla(f \circ G)$ found by hand?

Remark

A useful mnemonic for the chain rule uses differential notation. If we let x and y be the variables of f, and u and v be the variables of G, and we write z for $f \circ G$ and for f, and write the components of $G(u, v)$ as x and y, then the chain rule reads that

$$\frac{\partial f}{\partial u} = \frac{\partial f}{\partial x}\frac{\partial x}{\partial u} + \frac{\partial f}{\partial y}\frac{\partial y}{\partial u}$$

and that

$$\frac{\partial f}{\partial v} = \frac{\partial f}{\partial x}\frac{\partial x}{\partial v} + \frac{\partial f}{\partial y}\frac{\partial y}{\partial v},$$

so that the partials in the numerator and denominator again appear to 'cancel'.

Activity: Partial derivatives through a change of variables (U3-LO6)

Suppose that

$$z = x^2 + xy - y^2$$

that $x = e^{2s+t}$, and that $y = \ln(t/s)$. Compute

$$\frac{\partial z}{\partial s} \quad \text{and} \quad \frac{\partial z}{\partial t}.$$

First compute the partial derivatives of z in x and y, and the partial derivatives of x and y in t and s.

Activity: Partial derivatives through a change of variables (U3-LO6)

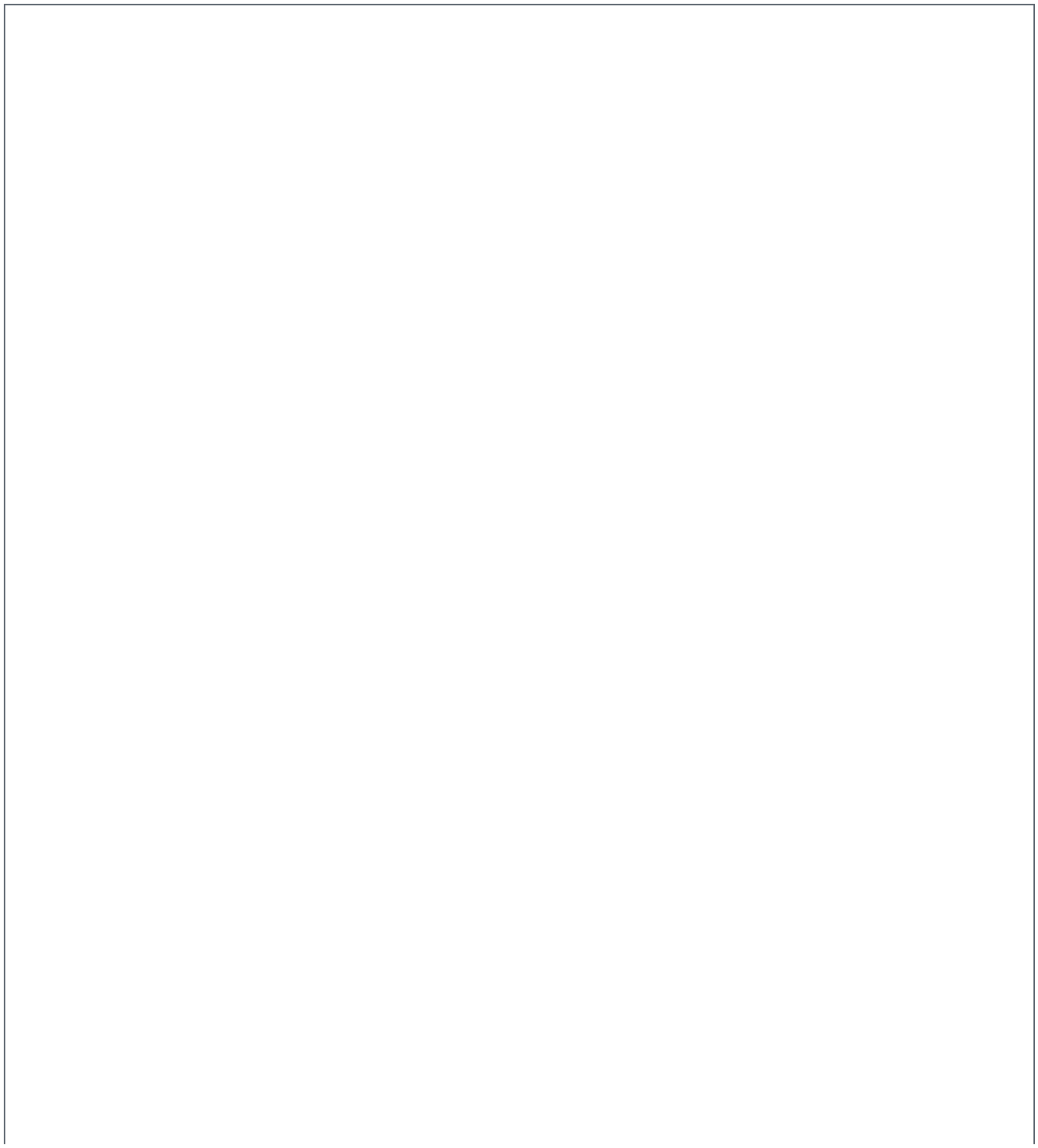
Suppose that

$$z = x^2 + xy - y^2$$

that $x = e^{2s+t}$, and that $y = \ln(t/s)$. Compute

$$\frac{\partial z}{\partial s} \quad \text{and} \quad \frac{\partial z}{\partial t}.$$

Now use the chain rule to compute the partial derivatives of z in s and t.



Generalized chain rule

Before multiplying Jacobian matrices, check the input and output dimensions of each map.

Theorem: Generalized Chain Rule

Suppose $F : \mathbb{R}^p \to \mathbb{R}^m$ and $G : \mathbb{R}^n \to \mathbb{R}^p$ are differentiable functions. Then $F \circ G : \mathbb{R}^n \to \mathbb{R}^m$ is differentiable, and

$$J_{F\circ G}(\mathbf{x}) = J_F(G(\mathbf{x}))J_G(\mathbf{x}),$$

where the right hand side is defined by matrix multiplications. Write $y_1, \dots, y_p$ for the input coordinates of F. Equivalently, for $1 \le i \le m$ and $1 \le j \le n$,

$$\frac{\partial(F_i \circ G)}{\partial x_j}(\mathbf{x}) = \sum_{k=1}^{p} \frac{\partial F_i}{\partial y_k}(G(\mathbf{x}))\frac{\partial G_k}{\partial x_j}(\mathbf{x}).$$

Why is this true?

Fix a base point $\mathbf{a}$ and write $\mathbf{b} = G(\mathbf{a})$. For a small input change $\mathbf{h}$, the local linear approximation of G gives

$$G(\mathbf{a} + \mathbf{h}) \approx \mathbf{b} + J_G(\mathbf{a})\mathbf{h}.$$

Thus the change in the input to F is approximately $J_G(\mathbf{a})\mathbf{h}$.
Near $\mathbf{b}$, the local linear approximation of F is

$$F(\mathbf{b} + \mathbf{k}) \approx F(\mathbf{b}) + J_F(\mathbf{b})\mathbf{k}.$$

Substitute the actual intermediate change $\mathbf{k} = G(\mathbf{a} + \mathbf{h}) - \mathbf{b}$, then use the approximation from G:

$$\begin{aligned}(F \circ G)(\mathbf{a} + \mathbf{h}) &\approx F(\mathbf{b}) + J_F(\mathbf{b})\big(G(\mathbf{a} + \mathbf{h}) - \mathbf{b}\big) \\ &\approx F(\mathbf{b}) + J_F(\mathbf{b})J_G(\mathbf{a})\mathbf{h}.\end{aligned}$$

We have obtained the local affine approximation

$$L(\mathbf{x}) = F(\mathbf{b}) + J_F(\mathbf{b})J_G(\mathbf{a})(\mathbf{x} - \mathbf{a}).$$

Its linear part sends $\mathbf{h}$ to $J_F(\mathbf{b})J_G(\mathbf{a})\mathbf{h}$. Reading off the matrix of this linear part gives

$$J_{F\circ G}(\mathbf{a}) = J_F(G(\mathbf{a}))J_G(\mathbf{a}).$$

The rightmost matrix acts first: $J_G(\mathbf{a})$ predicts the intermediate change, and $J_F(\mathbf{b})$ predicts the resulting output change. Differentiability ensures that the neglected errors are smaller than first order in $\mathbf{h}$, so they do not change this linear part.

Note

The generalized chain rule says that local linear maps compose. The matrix formulation records that composition by multiplying Jacobian matrices in the correct order.

The order matters. The rightmost Jacobian matrix acts first on the input change.

Note

If f is a scalar-valued function, then the generalized chain rule tells us that

$$J_{f\circ G} = (J_f \circ G)J_G.$$

Transposing and using $J_f = (\nabla f)^T$ gives

$$\nabla(f \circ G) = J_G^T(\nabla f \circ G),$$

as in the two-variable case.

Activity: The chain rule through a three-component map (U3-LO6)

Suppose $f(x, y, z) = 3x^2 - 2xy + 4z^2$, and that $G(u, v) = (e^u \sin v, e^u \cos v, e^u)$. Compute the partial derivatives of $f \circ G$.

Activity: Differentiating a loss through its predictions (U3-LO6)

The squared-error loss in From predictions to a scalar error function (U3-LO1, U2-LO1) is a composition:

$$\mathbf{w} \longmapsto R(\mathbf{w}) = X\mathbf{w} - \mathbf{y} \longmapsto q(R(\mathbf{w})), \qquad q(\mathbf{r}) = \|\mathbf{r}\|^2 = \mathbf{r}^T\mathbf{r}.$$

Thus $\mathscr{L} = q \circ R$. The fixed data are still $X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$ and $\mathbf{y} = (1, 2, 2)$.

1. Find $J_R(\mathbf{w})$ and $J_q(\mathbf{r})$, including their shapes. Which map is affine, and which is nonlinear?
2. Use the matrix chain rule to obtain $J_{\mathscr{L}}(\mathbf{w})$, and transpose it to obtain $\nabla\mathscr{L}(\mathbf{w})$. Evaluate at $\mathbf{w}_0 = (1, 1)$ and compare with How model weights change the loss (U3-LO3).
3. Let $\mathbf{w}(t) = (1, 1) + t(1, 1)$. Find $\frac{d}{dt}\mathscr{L}(\mathbf{w}(t))$ using the chain rule. Check by first substituting into the loss and then differentiating the resulting one-variable polynomial.

A tiny sigmoid block

A small neural-network block can be read as a composition of three steps: an affine map, a coordinatewise nonlinear rule, and another affine map.

Note: Sigmoid and its derivative

Let

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$

The derivative is

$$\sigma'(t) = \sigma(t)(1 - \sigma(t)).$$

When σ is applied to a vector, it is applied coordinate by coordinate.

Activity: A tiny sigmoid block (U3-LO5, U3-LO6, U3-LO7)

Let

$$W_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad W_2 = \begin{bmatrix} 1 & 0 & -1/2 \\ 0 & 1 & 1/2 \end{bmatrix},$$

and let $\mathbf{b}_1 = \mathbf{0}$ and $\mathbf{b}_2 = \mathbf{0}$.
Define

$$\mathbf{s} = W_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{u} = \sigma(\mathbf{s}), \qquad N(\mathbf{x}) = W_2\mathbf{u} + \mathbf{b}_2,$$

where σ is applied coordinatewise.
At the base point

$$\mathbf{a} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

do the following.

1. Compute the pre-activation vector $\mathbf{s}_\mathbf{a} = W_1\mathbf{a} + \mathbf{b}_1$.
2. Compute the hidden vector $\mathbf{u}_\mathbf{a} = \sigma(\mathbf{s}_\mathbf{a})$.
3. Compute $N(\mathbf{a})$.
4. Use $\sigma'(0) = 1/4$ to compute

$$D_\mathbf{a} = \operatorname{diag}\big(\sigma'((\mathbf{s}_\mathbf{a})_1), \sigma'((\mathbf{s}_\mathbf{a})_2), \sigma'((\mathbf{s}_\mathbf{a})_3)\big).$$

Activity: A tiny sigmoid block (U3-LO5, U3-LO6, U3-LO7) (continued)

5. Use the chain rule to compute

$$J_N(\mathbf{a}) = W_2 D_{\mathbf{a}} W_1.$$

6. Use

$$N(\mathbf{a} + \mathbf{h}) \approx N(\mathbf{a}) + J_N(\mathbf{a})\mathbf{h}$$

to predict the output change caused by a small input change $\mathbf{h}$.

Note: Why sigmoid here

The sigmoid function is smooth, so the Jacobian matrix of this block is defined at every input. Some neural-network models use nonsmooth activation rules. Those require extra case analysis. Here the point is the chain rule for Jacobian matrices.

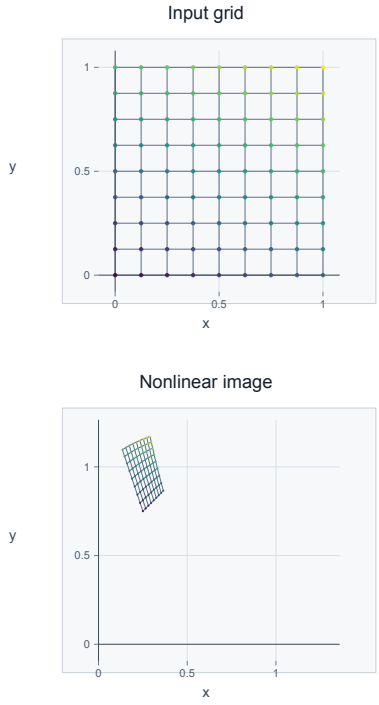


Figure. Square-grid visualization for a two-output sigmoid block.

Activity: A one-query softmax weighting map (U3-LO7, U1-LO8)

A single-query attention-style map can be viewed as a nonlinear vector-valued map.
Let

$$K = \begin{bmatrix} 3 & 0 \\ 0 & 3 \\ -3 & -3 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

For a query vector

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix},$$

define the score vector

$$\mathbf{s} = K\mathbf{q},$$

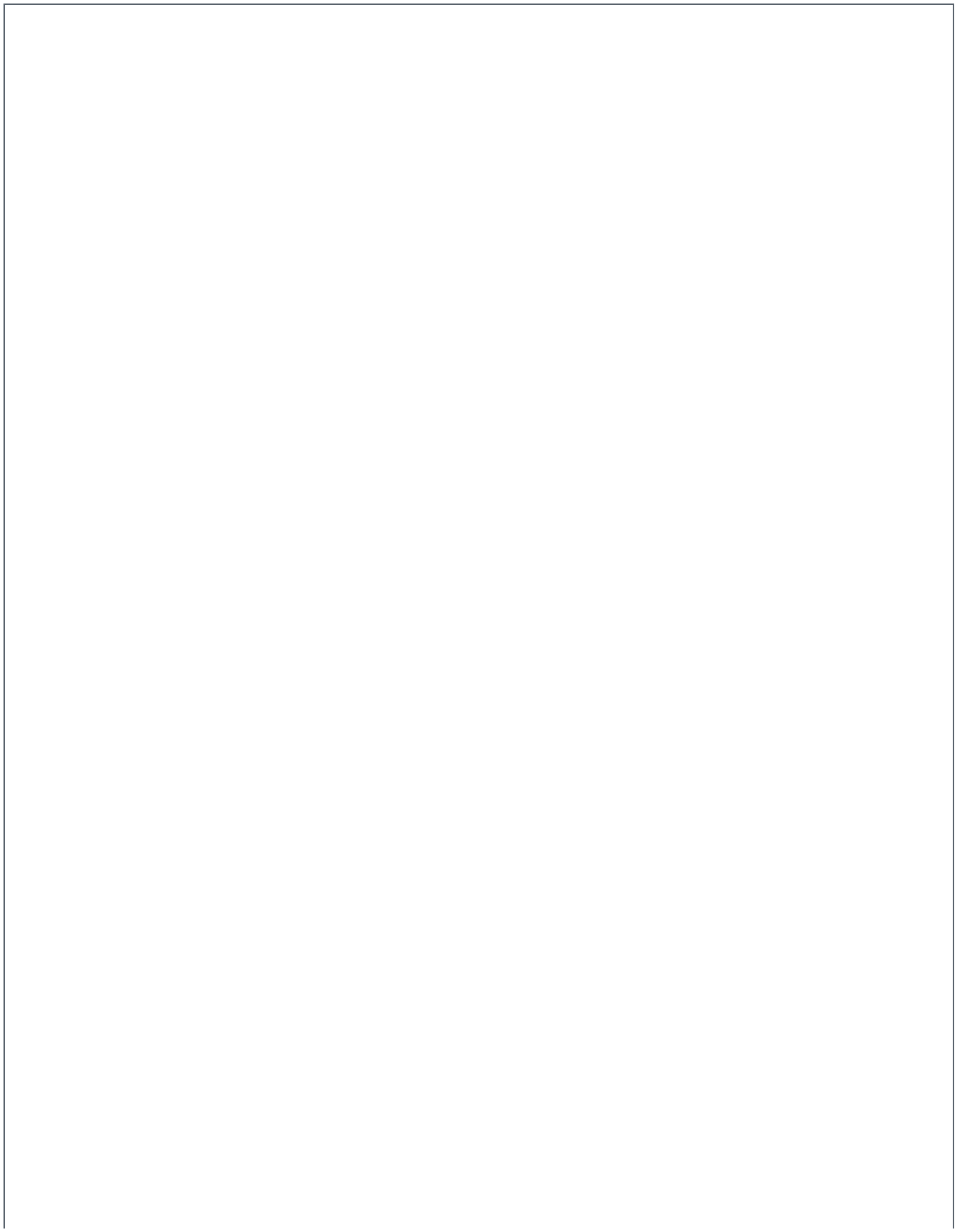
the softmax weight vector

$$\alpha(\mathbf{q}) = \mathrm{softmax}(\mathbf{s}),$$

and the output

$$F(\mathbf{q}) = V^T \alpha(\mathbf{q}).$$

1. Compute $\mathbf{s} = K\mathbf{q}$ in terms of q_1 and q_2.
2. What are the rows of V?
3. Explain why $F(\mathbf{q})$ is a weighted average of the rows of V.
4. Compute $\alpha(\mathbf{0})$ and $F(\mathbf{0})$.
5. Explain why F is not a linear map.
6. What operation makes F nonlinear?



Note: Softmax outputs stay in the value triangle

The output always lies in the triangle spanned by the three value vectors. The query vector does not choose fixed weights. It changes the scores, and the softmax rule changes all weights together.



Figure. Square-grid visualization for an attention-style softmax map.

Note: An attention-style nonlinear block

Unit 1 treated attention as three visible operations:

$$\text{dot-product scores} \longrightarrow \text{weights} \longrightarrow \text{weighted averages.}$$

For a sequence of token vectors stored as rows of a matrix X, fixed weight matrices produce

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

The score matrix is

$$S = QK^T.$$

The entry S_{ij} compares the query for token i with the key for token j. A weighting rule, such as row-wise softmax, converts each row of S into weights. Write the resulting attention-weight matrix as A_{att}. The attention output is

$$H = A_{\text{att}}V.$$

If A_{att} were fixed, then $V \mapsto A_{\text{att}}V$ would be a linear map: each row of H would be a weighted average of rows of V. In attention, A_{att} is not fixed. It is computed from X through Q, K, scores, and softmax. Thus the full rule $X \mapsto H$ is generally nonlinear.

The Unit 3 question is local. Near one fixed input X_0, a nonlinear rule can be approximated by a linear map. After the entries of X and H are arranged as vectors, that local linear map is represented by a Jacobian matrix.

Activity: What depends on the input? (U3-LO5, U3-LO7)

Assume the weight matrices W_Q, W_K, and W_V are fixed. The input is the token matrix X. Define

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

$$S = QK^T, \qquad A_{\text{att}} = \text{softmax}_{\text{rows}}(S), \qquad H = A_{\text{att}}V.$$

1. Which objects are fixed before the input X is chosen?
2. Which objects are computed from X?
3. Why is $X \mapsto XW_Q$ a linear map when W_Q is fixed?
4. Why does $S = QK^T$ depend on X in two places?
5. If A_{att} were fixed, what would $H = A_{\text{att}}V$ do to the rows of V?
6. In actual attention, why is the full rule $X \mapsto H$ not just one linear map?

Activity: What depends on the input? (U3-LO5, U3-LO7) (continued)

7. What Unit 3 object describes the local linear approximation near a fixed input X_0?

Warning: Specify the input

Before asking whether a rule is linear, first say what is variable and what is fixed. The rule $V \mapsto A_{\text{att}}V$ is linear when A_{att} is fixed. The attention rule $X \mapsto A_{\text{att}}(X)V(X)$ is generally nonlinear because the weights and values both depend on X.

Activity: Comparing square-grid visualizations (U3-LO1, U3-LO5, U3-LO7, U1-LO8)

This unit used three square-grid visualizations: the nonlinear shear from Corners are not enough (U3-LO1), the sigmoid block from A tiny sigmoid block (U3-LO5, U3-LO6, U3-LO7), and the attention-style softmax map from A one-query softmax weighting map (U3-LO7, U1-LO8).

Map A:

$$F\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \begin{bmatrix} x + y^2 \\ y \end{bmatrix}.$$

Map B: a two-output sigmoid block

$$\mathbf{s} = W_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{u} = \sigma(\mathbf{s}), \qquad N(\mathbf{x}) = W_2\mathbf{u} + \mathbf{b}_2.$$

Map C: an attention-style softmax map

$$F_{\text{att}}(\mathbf{q}) = V^T \operatorname{softmax}(K\mathbf{q}).$$

Activity: Comparing square-grid visualizations (U3-LO1, U3-LO5, U3-LO7, U1-LO8) (continued)

1. Which map agrees with a linear shear on the four corners but not at an edge midpoint?
2. Which map is built from affine maps with a coordinatewise sigmoid between them?
3. For which map is the local matrix computed by $W_2 D_{\mathrm{a}} W_1$?
4. Which map produces outputs inside the triangle spanned by value vectors?
5. For which map does the nonlinear step come from softmax weights that depend on the input?
6. In all three examples, what does the square-grid visualization show that four corners alone may miss?

